前言

本講義介紹知識蒸餾 knowledge distillation（KD）：把一個大模型學到的知識，移轉給一個小模型的訓練方法。 內容依據 Hinton、Vinyals、Dean 於 2015 年發表的論文〈Distilling the Knowledge in a Neural Network〉， 論文的公式、實驗設定與結果都完整列在本講義中，讀者不需要另外查閱原文。

本講義回答三個問題：

1. 為什麼需要把大模型的知識移轉給小模型。
2. 移轉時，小模型要學什麼、損失函數怎麼寫、梯度怎麼算。
3. 這個方法在手寫數字資料上的效果如何。

第 2 至 4 章用同一組自訂的小數字逐步計算，每一步都可以用計算機核對； 第 5 章用 Colab 程式重做論文的 MNIST 實驗。

怎麼讀本講義

章	內容	要動手的地方
1	為什麼要蒸餾	—
2	溫度與軟目標	2.3 節兩張計算表
3	蒸餾的損失函數	3.3 節計算表
4	梯度，以及高溫下的極限	4.2 節計算表
5	MNIST 實驗	Colab 跑一次
6	論文的其他結果	—
7	Colab 程式的使用方式	—
8	練習與解答	四題

術語對照

本講義沿用論文的術語，以便和原文對照。現今文獻多把笨重模型稱為老師模型 teacher model、蒸餾模型稱為學生模型 student model；目前查到最早明確這樣定義的是 J. Li、R. Zhao、J.-T. Huang、Y. Gong 於 Interspeech 2014 發表的〈Learning Small-Size DNN with Output-Distribution-Based Criteria〉：「we sometimes call the large-size DNN as the teacher DNN, and the small-size DNN as the student DNN」。

中文	英文	意義	本講義的符號
知識蒸餾	knowledge distillation（KD）	把笨重模型的知識移轉給適合部署的小模型的訓練方式	—
笨重模型	cumbersome model	為了從資料中學到規律而訓練的大模型：多個模型的組合，或以強正則化訓練的大型網路；現今多稱老師模型	其 logit 記為 v
蒸餾模型	distilled model	以軟目標與正確標籤訓練、承接笨重模型知識的小模型；使用時溫度為 1；現今多稱學生模型	其 logit 記為 z，機率記為 q
組合模型	ensemble	多個分別訓練的模型，把它們的預測取平均	—
正則化	regularization	訓練時限制模型、避免它過度貼合訓練資料（稱為過擬合）的方法，例如 dropout 或權重限制	—
dropout	dropout	訓練時每次隨機停用一部分神經元的正則化方法；可視為同時訓練許多共用權重的模型所組成的組合	—
移轉集	transfer set	用來訓練蒸餾模型的資料，可以是原本的訓練集，也可以是無標籤資料	—
類推	generalization	模型對沒見過的新資料做出正確判斷的能力，又稱泛化	—
logit	logit	softmax 的輸入：網路最後一層對每個類別算出的數值	z_i、v_i
softmax	softmax	把每個類別的 logit 轉成機率的函數，所有類別的機率加總為 1	—
溫度	temperature	softmax 中除以 logit 的常數；T 越大，機率分布越軟，通常 T = 1	T
軟目標	soft targets	笨重模型在高溫下產生的類別機率，保留錯誤類別之間的相對機率	p
硬目標	hard targets	正確標籤：只有正確類別為 1，其餘為 0	y
交叉熵	cross entropy	兩個機率分布之間差異的度量	C
比對 logit	matching logits	讓小模型的 logit 直接貼齊笨重模型的 logit；是高溫極限下的蒸餾	—
專家模型	specialist models	只負責一群容易混淆類別的模型，與通用模型 generalist model 合用	—

第 1 章　為什麼要蒸餾

1.1 訓練與部署的需求不同

許多昆蟲有幼蟲與成蟲兩種形態：幼蟲適合從環境中攝取養分，成蟲適合移動與繁殖。 論文以此比喻機器學習：訓練階段與部署階段的需求不同，卻常使用同一個模型。

	訓練階段	部署階段
資料量	很大，而且重複性高	一次處理一筆
時間限制	可以計算數天	要即時回應
計算資源	大量 GPU	手機或伺服器
適合的模型	大模型，或多個模型的組合	小模型

訓練時可以使用很大的模型，例如多個模型的組合 ensemble，或是用 Dropout 這類強正則化訓練的單一大模型， 因為這有助於從資料中學到規律。訓練完成後，再用另一種訓練方式，把大模型的知識移轉給適合部署的小模型。 這種訓練方式稱為**蒸餾**。本講義把大模型稱為**笨重模型**，小模型稱為**蒸餾模型**。

1.2 知識是什麼

一般認為一個模型的知識就是它的參數。依這個看法，模型的架構一改，知識就無法保留。

論文採用另一個看法：模型的知識是它學到的**輸入到輸出的對應**。依這個看法， 只要蒸餾模型對每一個輸入給出和笨重模型相近的輸出，知識就移轉過去了，兩者的架構不必相同。

1.3 錯誤答案的機率也有資訊

分類模型訓練時，目標是提高正確類別的機率；但訓練完成後，它也會給每個錯誤類別一個機率。 這些機率雖然小，大小卻不相同，反映模型怎麼看待輸入。論文舉了兩個例子：

輸入	被認為是	機率
一張 BMW 的照片	垃圾車	很小
一張 BMW 的照片	紅蘿蔔	比垃圾車小很多倍

輸入	被認為是 3	被認為是 7
第一種寫法的 2	10^{-6}	10^{-9}
第二種寫法的 2	10^{-9}	10^{-6}

第二個例子說明：第一種寫法的 2 比較像 3，第二種寫法的 2 比較像 7。這是資料之間的相似關係，只看正確答案得不到。大模型通常比小模型更懂得如何類推到新資料，蒸餾模型如果學到這些機率，也就學到笨重模型類推的方式。

問題在於 10^{-6} 與 10^{-9} 都太接近 0。用一般的交叉熵訓練蒸餾模型時，這兩個數字幾乎不影響損失，蒸餾模型學不到它們。 Caruana 等人的做法是讓蒸餾模型直接學笨重模型 softmax 之前的數值（logit），以平方差作為損失。 本論文提出的做法是提高 softmax 的**溫度**，第 4.3 節會說明，前者是後者的特例。

1.4 作者的回顧

論文第一作者 Hinton 在 2026 年 10 月的一次訪談中，回顧了蒸餾這個想法的由來 [5]，內容與本章一致：

1. 讓小模型學大模型的輸出，Caruana 等人在 2006 年就提出了，也就是第 1.3 節提到的做法；但當時很少人注意，Hinton 自己也沒有。

2. 他後來思考大模型時聯想到昆蟲：多數昆蟲有幼蟲與成蟲兩個階段，幼蟲只負責進食，從環境中攝取養分；成蟲負責移動與繁殖。
3. 對應到機器學習：要從資料中萃取資訊，最適合的是連結數目非常多的大模型；要同時服務十億人，則需要小得多的模型。蒸餾做的，就是把資訊從大模型移到小模型。

昆蟲	機器學習	本講義的名稱
幼蟲：攝取養分	訓練：從資料中萃取規律	笨重模型
成蟲：移動與繁殖	部署：同時回應大量使用者	蒸餾模型
由幼蟲變為成蟲	把知識移轉給小模型	蒸餾

論文發表十一年後，作者的說法仍是同一個比喻，可見這是理解蒸餾最核心的一點：訓練與部署的需求不同，所以用兩個不同的模型。

第 2 章　溫度與軟目標

2.1 溫度軟化的 softmax

神經網路最後一層輸出的數值稱為 logit，第 i 個類別的 logit 記為 z_i。softmax 把 logit 轉成機率 q_i。 論文的式 (1) 在 softmax 中加入溫度 T：

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \tag{1}$$

$T = 1$ 就是一般的 softmax。T 越大，各類別的機率越接近，機率分布越平均。

2.2 示範資料

本講義第 2 至 4 章使用同一組自訂的數字。輸入是一張手寫的 2，為了能手算，只取「2、3、7」三個類別。

類別	笨重模型 logit v	蒸餾模型 logit z	正確答案 y
2	8	4	1
3	4	2	0
7	0	2	0

笨重模型很確定這是 2，也認為 3 比 7 像；蒸餾模型分得出 2，但對 3 和 7 給出相同的 logit。這組數字是為了手算而自訂的，不是實際模型的輸出。

2.3 逐步計算

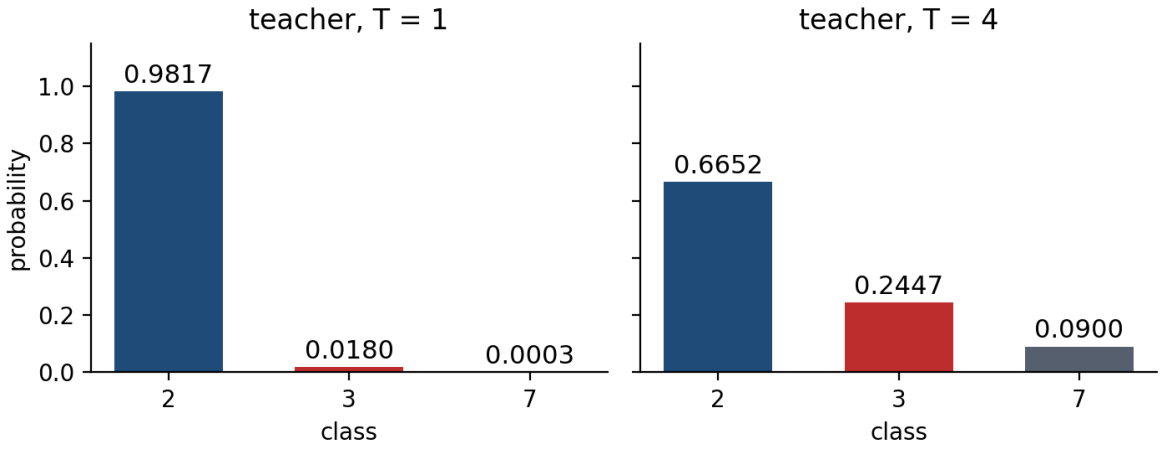
式 (1) 分四步計算：每個 logit 除以 T、取指數、加總、各自除以總和。

$T = 1$

類別	v	第 1 步 v/T	第 2 步 $\exp(v/T)$	第 4 步 $p = \exp \div$ 總和
2	8	8.0000	2980.9580	0.9817
3	4	4.0000	54.5982	0.0180
7	0	0.0000	1.0000	0.0003
第 3 步 總和			3036.5561	

$T = 4$

類別	v	第 1 步 v/T	第 2 步 $\exp(v/T)$	第 4 步 $p = \exp \div$ 總和
2	8	2.0000	7.3891	0.6652
3	4	1.0000	2.7183	0.2447
7	0	0.0000	1.0000	0.0900
第 3 步 總和			11.1073	



笨重模型在兩種溫度下的機率

$T = 1$ 時，3 和 7 的機率是 0.0180 與 0.0003，都接近 0；$T = 4$ 時變成 0.2447 與 0.0900。 兩者的比值等於 $\exp\big((4-0)/T\big)$：$T = 1$ 時是 $e^4 = 54.60$，$T = 4$ 時是 $e^1 = 2.72$。 提高溫度不改變類別的排序，只縮小機率之間的差距，使錯誤類別的機率大到足以影響訓練。

2.4 軟目標與硬目標

笨重模型在高溫下給出的機率稱為**軟目標**，本例在 $T = 4$ 時為 $p = (0.6652, 0.2447, 0.0900)$。 正確答案本身稱為**硬目標**，本例為 $y = (1, 0, 0)$。

論文指出，軟目標的熵較高時，每一筆訓練資料提供的資訊比硬目標多，各筆資料之間的梯度變異也比較小； 因此蒸餾模型通常可以用比笨重模型少得多的資料、較大的學習率訓練。

用來訓練蒸餾模型的資料稱為**移轉集**。移轉集可以是沒有標籤的資料，也可以直接使用原本的訓練集。 論文發現使用原本的訓練集效果很好，尤其是損失函數再加上一小項正確答案時，見第 3 章。

第 3 章　蒸餾的損失函數

3.1 兩個目標函數

蒸餾模型的損失由兩項加權平均組成：

- **軟目標項**：笨重模型的軟目標 p 與蒸餾模型的機率 q 的交叉熵。p 與 q 都在**同一個溫度** T 下計算。

$$C_{soft} = -\sum_i p_i \log q_i$$

- **硬目標項**：正確答案與蒸餾模型在 $T = 1$ 下的機率的交叉熵。因為 y 只有正確類別是 1，這一項只剩一個對數：

$$C_{hard} = -\log q_{\text{correct}}^{(T=1)}$$

兩項使用蒸餾模型的同一組 logit，差別只在溫度。總損失為

$$L = \alpha\, T^2\, C_{soft} + (1 - \alpha)\, C_{hard}$$

3.2 兩個係數

- α 是軟目標項的權重。論文指出，硬目標項的權重取得比軟目標項低很多，結果通常最好。本講義取 $\alpha = 0.9$。
- T^2：軟目標項產生的梯度大小與 $1/T^2$ 成正比（第 4.3 節驗證）。乘上 T^2 之後，實驗時改變溫度，兩項的相對比重也大致不變。

蒸餾模型訓練完成後，使用時溫度回到 $T = 1$。

3.3 逐步計算

第一步：蒸餾模型在 $T = 4$ 的機率 q

類別	z	$z/4$	$\exp(z/4)$	q
2	4	1.0000	2.7183	0.4519
3	2	0.5000	1.6487	0.2741
7	2	0.5000	1.6487	0.2741
總和			6.0157	

第二步：軟目標項 C_{soft}

類別	笨重模型 p	$\log q$	$p \times \log q$
2	0.6652	−0.7944	−0.5285
3	0.2447	−1.2944	−0.3168
7	0.0900	−1.2944	−0.1165
總和			−0.9618

$C_{soft} = -(-0.9618) = 0.9618$。log 是自然對數；各格四捨五入到小數第 4 位，總和以未進位的值計算。

第三步：蒸餾模型在 $T = 1$ 的機率與硬目標項 C_{hard}

類別	z	$\exp(z)$	$q^{(T=1)}$
2	4	54.5982	0.7870
3	2	7.3891	0.1065
7	2	7.3891	0.1065
總和		69.3763	

正確答案是 2，所以 $C_{hard} = -\log 0.7870 = 0.2395$。

第四步：總損失

項	算式	值
軟目標項	$0.9 \times 4^2 \times 0.9618$	13.8493
硬目標項	0.1×0.2395	0.0240
總損失 L	兩項相加	13.8732

第 4 章　梯度，以及高溫下的極限

4.1 式 (2)

訓練蒸餾模型時，需要損失對蒸餾模型 logit z_i 的梯度。軟目標項的梯度是論文的式 (2)：

$$\frac{\partial C_{soft}}{\partial z_i} = \frac{1}{T}\left(q_i - p_i\right) \tag{2}$$

推導如下。$q = \mathrm{softmax}(z/T)$，所以 $\log q_j = z_j/T - \log \sum_k \exp(z_k/T)$，對 z_i 微分得 $\frac{\partial \log q_j}{\partial z_i} = \frac{1}{T}(\delta_{ij} - q_i)$，其中 δ_{ij} 在 $i = j$ 時為 1，否則為 0。代入 C_{soft}：

$$\frac{\partial C_{soft}}{\partial z_i} = -\sum_j p_j \cdot \frac{1}{T}(\delta_{ij} - q_i) = \frac{1}{T}\left(q_i \sum_j p_j - p_i\right) = \frac{1}{T}(q_i - p_i)$$

最後一步用到 $\sum_j p_j = 1$。同樣的推導在 $T = 1$ 時給出硬目標項的梯度 $q_i^{(T=1)} - y_i$。

4.2 逐步計算

類別	$q - p$	$\div T$（式 2）	$\times \alpha T^2 = 14.4$	$q^{(T=1)} - y$	$\times (1-\alpha) = 0.1$	總梯度
2	−0.2134	−0.0533	−0.7682	−0.2130	−0.0213	−0.7895
3	0.0293	0.0073	0.1056	0.1065	0.0107	0.1163
7	0.1840	0.0460	0.6625	0.1065	0.0107	0.6732

梯度下降法沿著梯度的反方向更新參數：2 的 logit 會變大，7 的 logit 下降最多，3 只小幅下降。 蒸餾模型因此往笨重模型的看法「2 最像、3 其次、7 最不像」靠近。只用硬目標項時，3 與 7 的梯度相同（都是 0.1065），蒸餾模型學不到兩者的差別。

Colab 程式第 3 段用 TensorFlow 的自動微分 `GradientTape` 重算同一組數字，得到相同的損失 13.8732 與相同的總梯度。

4.3 為什麼乘 T^2

同一組 logit，在三種溫度下計算式 (2)：

T	式 (2) 梯度的最大絕對值	乘 T^2 後
1	0.194704	0.1947
4	0.053345	0.8535
20	0.002637	1.0547

T 從 4 升到 20，梯度縮小約 20 倍，接近 $(20/4)^2 = 25$ 倍；乘上 T^2 之後是 0.85 與 1.05，大小相近。 $1/T^2$ 的比例是下一節高溫近似的結果，$T = 1$ 不在這個近似的範圍內。

4.4 高溫下等於比對 logit

當 x 很小時，$\exp(x) \approx 1 + x$。若溫度遠大於 logit，z_i/T 與 v_i/T 都很小，式 (2) 可近似為論文的式 (3)：

$$\frac{\partial C_{soft}}{\partial z_i} \approx \frac{1}{T}\left(\frac{1 + z_i/T}{N + \sum_j z_j/T} - \frac{1 + v_i/T}{N + \sum_j v_j/T}\right) \tag{3}$$

N 是類別數。再假設笨重模型與蒸餾模型的 logit 各自減去平均值，即 $\sum_j z_j = \sum_j v_j = 0$，式 (3) 化簡為式 (4)：

$$\frac{\partial C_{soft}}{\partial z_i} \approx \frac{1}{NT^2}\left(z_i - v_i\right) \tag{4}$$

這正是最小化 $\frac{1}{2}(z_i - v_i)^2$ 的梯度乘上常數 $\frac{1}{NT^2}$。也就是說，**高溫時的蒸餾，等於讓蒸餾模型的 logit 直接貼齊笨重模型的 logit**。式 (4) 也說明了第 4.3 節的 $1/T^2$。

以本例驗證：減去平均後 $z = (1.3333, -0.6667, -0.6667)$，$v = (4, 0, -4)$，$N = 3$，$T = 20$。

類別	式 (2) 精確值	式 (4) 近似值
2	−0.00229	−0.00222
3	−0.00034	−0.00056
7	0.00264	0.00278

溫度較低時，式 (2) 對「比平均值負很多的 logit」幾乎不要求貼齊。這可能是好事：這些 logit 在笨重模型訓練時幾乎不受損失函數約束，可能含有很多雜訊；也可能是壞事：它們也可能帶有笨重模型的知識。論文的實驗發現，蒸餾模型太小、學不下笨重模型全部的知識時，中間的溫度最好，表示忽略這些很負的 logit 是有幫助的（第 5.4 節）。

第 5 章　MNIST 實驗

5.1 論文的設定與結果

MNIST 是 28×28 灰階的手寫數字影像，訓練集 60,000 張，測試集 10,000 張。論文第 3 節的設定：

	架構	正則化	測試集答錯張數
笨重模型	兩個隱藏層，各 1,200 個 ReLU 神經元	Dropout、權重限制，輸入影像隨機平移最多 2 個像素	67
蒸餾模型，只用硬目標	兩個隱藏層，各 800 個 ReLU 神經元	無	146
蒸餾模型，蒸餾 $T = 20$	同上	只有蒸餾的軟目標	74

蒸餾使蒸餾模型的答錯張數從 146 降到 74，接近笨重模型的 67。笨重模型的訓練資料經過平移，蒸餾模型的移轉集沒有平移；蒸餾模型卻學到了笨重模型處理平移影像的方式，因為這項資訊包含在軟目標中。

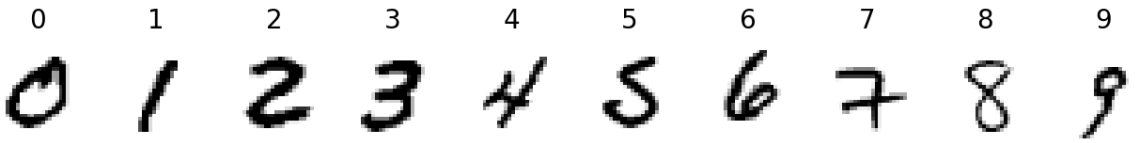
論文另外報告兩項結果：

1. 蒸餾模型每層有 300 個以上神經元時，8 以上的溫度結果相近；蒸餾模型每層縮小到 30 個時，2.5 到 4 的溫度明顯比更高或更低的溫度好。
2. 移轉集拿掉所有的 3，蒸餾模型從沒看過 3。蒸餾模型仍只答錯 206 張，其中 133 張是測試集 1,010 張 3 中的錯誤。答錯的 3 大多是因為 3 這一類的偏差值學得太低；把 3 的偏差值加 3.5（這個值以測試集挑選），答錯降為 109 張，其中 3 只錯 14 張，3 的答對率 98.6%。若移轉集只保留 7 和 8，蒸餾模型的測試錯誤率為 47.3%；把 7 和 8 的偏差值減 7.6 後降為 13.2%。

5.2 本講義的重做

本講義的 Colab 程式重做上述三項結果。為了在 Colab 免費版數分鐘內完成，資料與蒸餾模型都縮小：

	論文	本講義
訓練資料	60,000 張	10,000 張，0 到 9 各 1,000 張
測試資料	10,000 張	同一個測試集 10,000 張，其中 3 有 1,010 張
笨重模型	1200-1200，Dropout，平移 2 像素	1200-1200，Dropout，平移 2 像素
蒸餾模型	800-800；另試 30-30	100-100；另試 30-30
溫度	20，另試多種	1、2、4、8、20
α	硬目標權重取低	0.9



MNIST 訓練影像

以下數字是 2026 年 10 月 22 日在本機 CPU 執行 Colab 程式的結果（Python 3.13、TensorFlow 2.21、Keras 3.13）。同一份程式在不同次執行時，答錯張數可能不同（例如笨重模型實測出現過 203 與 222 兩種結果），但三項結果的趨勢不變。

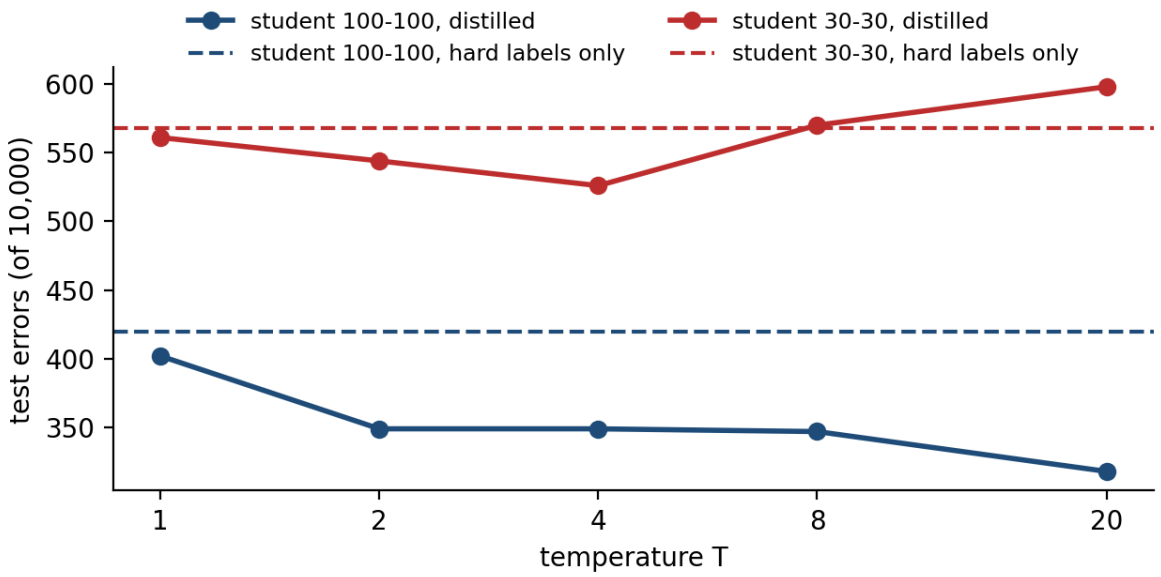
5.3 結果一：蒸餾模型答錯較少

模型	測試集答錯張數
笨重模型 1200-1200	203
小網路 100-100，只用硬目標	420
小網路 100-100，蒸餾 $T = 20$	318

蒸餾模型與笨重模型之間的差距，由 217 張縮小為 115 張。

5.4 結果二：蒸餾模型很小時，溫度取中間值

T	小網路 100-100	小網路 30-30
只用硬目標	420	568
1	402	561
2	349	544
4	349	526
8	347	570
20	318	598



答錯張數與溫度

小網路 100-100 在所有溫度下都比只用硬目標少錯，溫度越高越好。小網路 30-30 在 $T = 4$ 最好，$T = 20$ 反而比只用硬目標多錯。 這與論文的觀察一致：溫度越高，越接近第 4.4 節的比對 logit，連非常負的 logit 也要貼齊，容量太小的蒸餾模型學不來。

5.5 結果三：蒸餾模型從沒看過 3

移轉集拿掉所有的 3，剩 9,000 張。小網路 100-100。

模型	總共答錯	其中 3 答錯	3 的答對率
只用硬目標	1372	1010 / 1010	0.0%
蒸餾 $T = 8$	646	378 / 1010	62.6%
蒸餾 $T = 8$，3 的 logit 加 3.0	430	84 / 1010	91.7%

只用硬目標的小網路，訓練時沒有任何一張的答案是 3，不可能答出 3。 蒸餾模型雖然沒看過 3，笨重模型對其他數字給出的軟目標中，仍包含「這張有幾分像 3」的資訊，因此蒸餾模型認得出一半以上的 3； 加上偏差值修正後，答對率超過九成。加多少是用測試集挑選的，與論文相同，所以這個數字偏樂觀。

第 6 章　論文的其他結果

本章的數字取自論文，本講義沒有重做。

6.1 語音辨識

論文以 Android 語音搜尋所用的聲學模型（8 個隱藏層、每層 2,560 個神經元、輸出 14,000 個類別、約 8,500 萬個參數）為基準， 獨立訓練 10 個相同架構、不同初始值的模型，再把 10 個模型的組合蒸餾成一個同樣大小的模型。

系統	測試框架準確率	詞錯誤率 WER
單一模型	58.9%	10.9%
10 個模型的組合	61.1%	10.7%
蒸餾的單一模型	60.8%	10.7%

10 個模型的組合帶來的進步，超過 80% 移轉到了單一模型。

6.2 軟目標可以當作正則化

同一個語音模型只用 3% 的訓練資料（約 2,000 萬筆）訓練：

訓練方式	訓練資料	訓練框架準確率	測試框架準確率
硬目標	100%	63.4%	58.9%
硬目標	3%	67.3%	44.5%
軟目標	3%	65.4%	57.0%

只用硬目標時嚴重過擬合，必須提前停止；改用全部資料訓練出的模型所提供的軟目標，3% 的資料就能接近全部資料的結果，而且不需要提前停止。

6.3 專家模型

在 1 億張影像、15,000 個類別的 JFT 資料集上，論文另訓練 61 個專家模型 specialist models。每個專家負責一群容易混淆的類別（例如多種橋、多款車），其餘類別併成一個「其他」類。專家以原模型的權重初始化，可以各自獨立、平行訓練，數天內完成。與原模型合用時，測試準確率由 25.0% 提高到 26.1%；只看專家負責的類別，由 43.1% 提高到 45.9%。

第 7 章　Colab 程式的使用方式

程式 `knowledge_distillation.ipynb` 在 Colab 開啟後，由上往下逐格執行即可。程式全文列在本講義的附錄。共八段：

段	內容	對應本講義
1	載入資料	5.2 節
2	溫度軟化的 softmax 手算例	2.3 節
3	損失與梯度手算例，並以自動微分核對	3.3、4.2、4.3、4.4 節
4	訓練笨重模型	5.2 節
5	小網路 100-100 的溫度比較	5.3、5.4 節
6	小網路 30-30 的溫度比較	5.4 節
7	蒸餾模型從沒看過 3	5.5 節
8	結果整理	—

資料檔 `kd_mnist.npz`（約 3.3 MB）有兩種取得方式，程式會自動判斷：

1. 把本教材提供的 `kd_mnist.npz` 拖到 Colab 左側的「檔案」區，放在預設資料夾 `/content`。
2. 左側沒有這個檔時，程式從教材的 GitHub repo 下載同一個檔。

在 CPU 上全部執行約需數分鐘；改用 GPU（「執行階段 → 變更執行階段類型 → T4 GPU」）會更快。

第 8 章　練習與解答

練習一　$T = 2$ 時，笨重模型 $v = (8, 4, 0)$ 的軟目標是多少？依第 2.3 節的四步計算。

練習二　蒸餾模型 z 改成 $(8, 4, 0)$，和笨重模型相同，$T = 4$。式 (2) 的梯度是多少？為什麼？

練習三　在 Colab 第 7 段改成拿掉數字 7，重做一次，記錄 7 的答對率，以及加偏差值後的答對率。

練習四　依第 1.4 節，要同時服務十億人時，為什麼不直接部署笨重模型？用第 1.1 節表格中的哪兩列回答？

解答一　$v/2 = (4, 2, 0)$；取指數得 $(54.5982, 7.3891, 1.0000)$；總和 62.9873； 軟目標 $p = (0.8668, 0.1173, 0.0159)$。比 $T = 1$ 平均，比 $T = 4$ 集中。

解答二　蒸餾模型與笨重模型的 logit 相同，所以在同一溫度下 $q = p$，式 (2) 的 $q_i - p_i$ 全為 0，軟目標項的梯度為 $(0, 0, 0)$：蒸餾模型已經完全貼齊笨重模型，軟目標項不再推動它。 總梯度只剩硬目標項 $0.1 \times (q^{(T=1)} - y) = 0.1 \times (0.9817 - 1,\ 0.0180,\ 0.0003) = (-0.0018, 0.0018, 0.0000)$。

解答三　結果依每次訓練而不同，沒有固定答案。記錄時同時寫下總共答錯張數、7 答錯張數，以及加了多少偏差值。

解答四　用表格中「時間限制」與「計算資源」兩列回答。部署階段要即時回應，而且只能使用手機或伺服器的計算資源；笨重模型需要大量 GPU，回應太慢、成本也太高。所以先用大模型從資料中學到規律，再蒸餾成小模型部署。

附錄　程式全文

以下是 Colab 程式 `knowledge_distillation.ipynb` 的全部程式碼，依執行順序排列；標題對應程式的各段。

1. 載入資料

```
import os
import urllib.request

import numpy as np

np.set_printoptions(suppress=True)  # 小數不用科學記號，和簡報上的寫法一致

DATA = 'kd_mnist.npz'
URL = ('https://raw.githubusercontent.com/pychang-ai/115-1_DLMA/main/'
       'topics/knowledge-distillation/kd_mnist.npz')

if os.path.exists(DATA):
    print('使用已上傳的', DATA)
else:
    print('左側沒有', DATA, '，從教材的 GitHub repo 下載')
    urllib.request.urlretrieve(URL, DATA)

d = np.load(DATA)
x_train = d['x_train'].reshape(-1, 784).astype('float32') / 255.0
y_train = d['y_train'].astype('int32')
x_test = d['x_test'].reshape(-1, 784).astype('float32') / 255.0
y_test = d['y_test'].astype('int32')
print('訓練', x_train.shape, '測試', x_test.shape)
print('訓練集各數字張數', np.bincount(y_train))
print('測試集各數字張數', np.bincount(y_test))
```

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 10, figsize=(12, 1.6))
for k, ax in enumerate(axes):
    i = np.where(y_train == k)[0][0]
    ax.imshow(x_train[i].reshape(28, 28), cmap='gray_r')
    ax.set_title(str(k))
    ax.axis('off')
plt.show()
```

2. 溫度軟化的 softmax

```
def softmax_T(z, T):
    """論文式 (1)：溫度 T 的 softmax，逐步印出每一步。"""
    z = np.asarray(z, dtype='float64')
    a = z / T
    e = np.exp(a)
    s = e.sum()
    return a, e, s, e / s


v = [8.0, 4.0, 0.0]
for T in [1, 4]:
    a, e, s, p = softmax_T(v, T)
    print(f'T = {T}')
    print('  第 1 步 z/T      ', np.round(a, 4))
    print('  第 2 步 exp(z/T) ', np.round(e, 4))
    print('  第 3 步 加總      ', round(s, 4))
    print('  第 4 步 機率      ', np.round(p, 4))
    print(f'  「3」與「7」的機率比 {p[1] / p[2]:.2f}')
```

```
labels = ['2', '3', '7']
fig, axes = plt.subplots(1, 2, figsize=(8, 2.8), sharey=True)
for ax, T in zip(axes, [1, 4]):
    p = softmax_T(v, T)[3]
    ax.bar(labels, p, color=['#1F4E79', '#C03030', '#59636E'])
    for k, val in enumerate(p):
        ax.text(k, val + 0.02, f'{val:.4f}', ha='center')
    ax.set_title(f'teacher softmax, T = {T}')
    ax.set_ylim(0, 1.1)
plt.show()
```

3. 蒸餾的損失函數與梯度

```
T, alpha = 4.0, 0.9
z = np.array([4.0, 2.0, 2.0])
p = softmax_T(v, T)[3]          # 笨重模型的軟目標
q = softmax_T(z, T)[3]          # 蒸餾模型在 T = 4
q1 = softmax_T(z, 1)[3]         # 蒸餾模型在 T = 1
y = np.array([1.0, 0.0, 0.0])   # 正確答案是 2

c_soft = -(p * np.log(q)).sum()
c_hard = -(y * np.log(q1)).sum()
loss = alpha * T ** 2 * c_soft + (1 - alpha) * c_hard
print('笨重模型軟目標 p      ', np.round(p, 4))
print('蒸餾模型 q（T=4）     ', np.round(q, 4))
print('蒸餾模型 q（T=1）     ', np.round(q1, 4))
print(f'C_soft = {c_soft:.4f}   C_hard = {c_hard:.4f}')
print(f'L = {alpha} × {T:g}² × {c_soft:.4f} + {1 - alpha:.1f} × {c_hard:.4f} = {loss:.4f}')

g_soft = (q - p) / T            # 式 (2)
g_hard = q1 - y                 # 一般交叉熵的梯度
g = alpha * T ** 2 * g_soft + (1 - alpha) * g_hard
print()
print('式 (2) 軟目標梯度 (q - p)/T ', np.round(g_soft, 4))
print('硬目標梯度 q1 - y           ', np.round(g_hard, 4))
print('總梯度                       ', np.round(g, 4))
```

```
import tensorflow as tf

zt = tf.Variable(z)
with tf.GradientTape() as tape:
    qt = tf.nn.log_softmax(zt / T)
    q1t = tf.nn.log_softmax(zt)
    L = alpha * T ** 2 * -tf.reduce_sum(p * qt) + (1 - alpha) * -tf.reduce_sum(y * q1t)
print('自動微分的損失', round(float(L), 4))
print('自動微分的梯度', np.round(tape.gradient(L, zt).numpy(), 4))
```

```
for t in [1, 4, 20]:
    gs = (softmax_T(z, t)[3] - softmax_T(v, t)[3]) / t
    print(f'T = {t:>2}  梯度最大絕對值 {np.abs(gs).max():.6f}   乘 T² 後 {np.abs(gs).max() * t * t:.4f}')
```

```
N, T = 3, 20.0
zc, vc = z - z.mean(), np.array(v) - np.mean(v)
exact = (softmax_T(zc, T)[3] - softmax_T(vc, T)[3]) / T
approx = (zc - vc) / (N * T ** 2)
print('減去平均後  z =', np.round(zc, 4), '  v =', np.round(vc, 4))
print('式 (2) 精確值', np.round(exact, 5))
print('式 (4) 近似值', np.round(approx, 5))
```

4. 訓練笨重模型

```
import keras
from keras import layers

keras.utils.set_random_seed(1)


def mlp(units, regularize=False, name='mlp'):
    """兩個隱藏層的全連接網路，輸出 10 個 logit。regularize=True 時加平移與 Dropout。"""
    inp = keras.Input((784,))
    x = inp
    if regularize:
        x = layers.Reshape((28, 28, 1))(x)
        x = layers.RandomTranslation(2 / 28, 2 / 28, fill_mode='constant')(x)
        x = layers.Flatten()(x)
        x = layers.Dropout(0.2)(x)
    for _ in range(2):
        x = layers.Dense(units, activation='relu')(x)
        if regularize:
            x = layers.Dropout(0.5)(x)
    out = layers.Dense(10)(x)
    return keras.Model(inp, out, name=name)


def test_errors(model, bias=None):
    """回傳測試集答錯張數與 logit。bias 是加在各類別 logit 上的偏移量。"""
    logits = model.predict(x_test, batch_size=2000, verbose=0)
    if bias is not None:
        logits = logits + bias
    return int((logits.argmax(1) != y_test).sum()), logits


teacher = mlp(1200, regularize=True, name='teacher')
teacher.compile(keras.optimizers.Adam(1e-3),
                keras.losses.SparseCategoricalCrossentropy(from_logits=True))
teacher.fit(x_train, y_train, batch_size=128, epochs=30, verbose=0)
teacher_err, _ = test_errors(teacher)
print(f'笨重模型：參數 {teacher.count_params():,} 個，測試集 10,000 張答錯 {teacher_err} 張')

# 笨重模型在移轉集（就是訓練集）上的 logit，之後每個蒸餾模型都用這一份
v_train = teacher.predict(x_train, batch_size=2000, verbose=0)
```

5. 蒸餾模型：只用硬目標，與蒸餾比較

```
def train_student(units, T, alpha, mask=None, epochs=30, seed=2):
    """依 L = α T² C_soft + (1 − α) C_hard 訓練蒸餾模型。mask 用來從移轉集拿掉部分資料。"""
    keras.utils.set_random_seed(seed)
    student = mlp(units, name='student')
    opt = keras.optimizers.Adam(1e-3)
    x, yy, vv = (x_train, y_train, v_train) if mask is None else \
        (x_train[mask], y_train[mask], v_train[mask])
    ds = tf.data.Dataset.from_tensor_slices((x, yy, vv)).shuffle(len(x), seed=seed).batch(128)

    @tf.function
    def step(xb, yb, vb):
        with tf.GradientTape() as tape:
            zb = student(xb, training=True)
            pb = tf.nn.softmax(vb / T)                                    # 笨重模型的軟目標
            c_soft = -tf.reduce_mean(tf.reduce_sum(pb * tf.nn.log_softmax(zb / T), 1))
            c_hard = tf.reduce_mean(tf.nn.sparse_softmax_cross_entropy_with_logits(
                labels=yb, logits=zb))
            loss = alpha * T * T * c_soft + (1 - alpha) * c_hard
        grads = tape.gradient(loss, student.trainable_variables)
        opt.apply_gradients(zip(grads, student.trainable_variables))

    for _ in range(epochs):
        for xb, yb, vb in ds:
            step(xb, yb, vb)
    return student


TEMPS = [1, 2, 4, 8, 20]
results = {}
s = train_student(100, 1.0, 0.0)
results[(100, 'hard')] = test_errors(s)[0]
print(f'小網路 100：只用硬目標，答錯 {results[(100, "hard")]} 張')
for t in TEMPS:
    s = train_student(100, float(t), 0.9)
    results[(100, t)] = test_errors(s)[0]
    print(f'小網路 100：蒸餾 T = {t:>2}，答錯 {results[(100, t)]} 張')
```

6. 更小的網路：溫度要取中間值

```
s = train_student(30, 1.0, 0.0)
results[(30, 'hard')] = test_errors(s)[0]
print(f'小網路 30：只用硬目標，答錯 {results[(30, "hard")]} 張')
for t in TEMPS:
    s = train_student(30, float(t), 0.9)
    results[(30, t)] = test_errors(s)[0]
    print(f'小網路 30：蒸餾 T = {t:>2}，答錯 {results[(30, t)]} 張')
```

7. 蒸餾模型從沒看過數字 3

```
mask = y_train != 3
print('移轉集張數', int(mask.sum()), '，其中數字 3：', int((y_train[mask] == 3).sum()), '張')


def threes_wrong(logits):
    return int(((logits.argmax(1) != y_test) & (y_test == 3)).sum())


for name, t, a in [('只用硬目標', 1.0, 0.0), ('蒸餾 T = 8', 8.0, 0.9)]:
    s = train_student(100, t, a, mask=mask)
    err, logits = test_errors(s)
    best = None
    for b in np.arange(0, 15.01, 0.25):
        shift = np.zeros(10, 'float32')
        shift[3] = b
        e = int(((logits + shift).argmax(1) != y_test).sum())
        if best is None or e < best[0]:
            best = (e, b, threes_wrong(logits + shift))
    results[('no3', name)] = (err, threes_wrong(logits), best)
    print(f'{name}：答錯 {err} 張，其中 3 錯 {threes_wrong(logits)} / 1010')
    print(f'    3 的 logit 加 {best[1]:.2f} 後：答錯 {best[0]} 張，其中 3 錯 {best[2]} / 1010'
          f'（3 答對 {100 * (1 - best[2] / 1010):.1f}%）')
```

8. 結果整理

```
print(f'{"模型":<22}{"測試集答錯張數":>10}')
print(f'{"笨重模型 1200-1200":<22}{teacher_err:>10}')
for units in [100, 30]:
    print(f'{"小網路 %d：只用硬目標" % units:<22}{results[(units, "hard")]:>10}')
    best_t = min(TEMPS, key=lambda t: results[(units, t)])
    print(f'{"小網路 %d：蒸餾最佳 T=%d" % (units, best_t):<22}{results[(units, best_t)]:>10}')

fig, ax = plt.subplots(figsize=(7, 3.2))
for units, color in [(100, '#1F4E79'), (30, '#C03030')]:
    ax.plot(TEMPS, [results[(units, t)] for t in TEMPS], 'o-', color=color,
            label=f'student {units}-{units}, distilled')
    ax.axhline(results[(units, 'hard')], color=color, ls='--', label=f'student {units}-{units}, hard labels only')
ax.set_xscale('log')
ax.set_xticks(TEMPS)
ax.set_xticklabels([str(t) for t in TEMPS])
ax.set_xlabel('temperature T')
ax.set_ylabel('test errors (of 10,000)')
ax.legend(fontsize=8)
plt.show()
```

參考文獻

1. G. Hinton, O. Vinyals, J. Dean, “Distilling the Knowledge in a Neural Network,” arXiv:1503.02531, 2015.
2. C. Buciluǎ, R. Caruana, A. Niculescu-Mizil, “Model compression,” in *Proc. 12th ACM SIGKDD*, 2006, pp. 535–541.
3. Y. LeCun, C. Cortes, C. J. C. Burges, The MNIST database of handwritten digits.
4. J. Li, R. Zhao, J.-T. Huang, Y. Gong, “Learning small-size DNN with output-distribution-based criteria,” in *Proc. Interspeech*, 2014, pp. 1910–1914.
5. G. Hinton，訪談，2026 年 10 月，08:29–10:32。影片：YouTube「AI 101」頻道，〈AI 教父 Hinton 給 Dario 打勇氣 A+：說要放慢，大概會傷到 Anthropic 上市〉，https://www.youtube.com/watch?v=4IUvlEFoDK8 。